

EXPERIMENT 5

AIM:

Create SQL views for department salary summary and employee hierarchy. Test updatability of views. Implement a recursive CTE to display reporting chains.

1. Add Manager Information to Employee Table

Our original Employee table does not have a reporting/manager relationship. Add a Manager_ID column.

```
ALTER TABLE Employee
ADD Manager_ID INT NULL;
```

```
ALTER TABLE Employee
ADD CONSTRAINT fk_manager
FOREIGN KEY (Manager_ID) REFERENCES Employee(Emp_ID);
```

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0	Records: 0 Duplicates: 0 Warnings: 0	10	0	0
0	30	0	Records: 30 Duplicates: 0 Warnings: 0	2	0	0

Now assign managers. The top-level managers have NULL.

```
UPDATE Employee
SET Manager_ID = CASE
  WHEN Emp_ID IN (1, 2, 3, 4, 5, 6) THEN NULL
  WHEN Emp_ID IN (7, 8, 9, 10, 11, 12) THEN 8
  WHEN Emp_ID IN (13, 14, 15, 16, 17, 18) THEN 18
  WHEN Emp_ID IN (19, 20, 21, 22, 23, 24) THEN 21
  WHEN Emp_ID IN (25, 26, 27, 28, 29, 30) THEN 27
END;
```

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	30	0	Rows matched: 30 Changed: 24 Warnings: 0	34	0	24

You can check the hierarchy using:

```
SELECT Emp_ID, Emp_Name, Manager_ID
FROM Employee
ORDER BY Emp_ID;
```

Emp_ID	Emp_Name	Manager_ID
1	Aarav Sharma	null
2	Riya Gupta	null
3	Arjun Verma	null
4	Ananya Singh	null
5	Karan Mehta	null
6	Priya Sharma	null
7	Neha Kapoor	8
8	Rahul Jain	8
9	Simran Kaur	8
10	Aditya Roy	8
11	Megha Joshi	8
12	Vivek Malhotra	8
13	Ishita Rao	18
14	Manish Kumar	18

15	Pooja Agarwal	18
16	Rohit Bansal	18
17	Kavya Nair	18
18	Sahil Gupta	18
19	Tanya Singh	21
20	Yash Verma	21
21	Nisha Patel	21
22	Aman Khan	21
23	Shreya Das	21
24	Dev Sharma	21
25	Akash Yadav	27
26	Sneha Roy	27
27	Varun Singh	27
28	Komal Gupta	27
29	Nitin Sharma	27
30	Aditi Mehta	27

2. View for Department Salary Summary

Create a view showing the number of employees, total salary, average salary, minimum salary, and maximum salary for every department.

CREATE VIEW Department_Salary_Summary AS

SELECT

**d.Dept_ID, d.Dept_Name,
COUNT(e.Emp_ID) AS Total_Employees,
SUM(e.Salary) AS Total_Salary,
AVG(e.Salary) AS Average_Salary,
MIN(e.Salary) AS Minimum_Salary,
MAX(e.Salary) AS Maximum_Salary**

FROM Department d

LEFT JOIN Employee e

ON d.Dept_ID = e.Dept_ID

GROUP BY d.Dept_ID, d.Dept_Name;

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

Display the view

SELECT *

FROM Department_Salary_Summary;

Dept_ID	Dept_Name	Total_Employees	Total_Salary	Average_Salary	Minimum_Salary	Maximum_Salary
1	IT	6	338000.00	56333.333333	45000.00	70000.00
2	HR	6	330000.00	55000.000000	42000.00	72000.00
3	Finance	6	393000.00	65500.000000	52000.00	80000.00
4	Marketing	6	319000.00	53166.666667	44000.00	62000.00
5	Operations	6	338000.00	56333.333333	46000.00	69000.00

3. Employee Hierarchy View

Create a view showing each employee and their immediate manager.

CREATE VIEW Employee_Hierarchy AS

SELECT

**e.Emp_ID,
e.Emp_Name AS Employee_Name,
e.Job_Title, e.Dept_ID, e.Manager_ID,
m.Emp_Name AS Manager_Name**

FROM Employee e

LEFT JOIN Employee m

ON e.Manager_ID = m.Emp_ID;

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

Display the hierarchy

SELECT *

FROM Employee_Hierarchy;

Emp_ID	Employee_Name	Job_Title	Dept_ID	Manager_ID	Manager_Name
1	Aarav Sharma	Developer	1	null	null
2	Riya Gupta	Developer	1	null	null
3	Arjun Verma	Tester	1	null	null
4	Ananya Singh	Team Lead	1	null	null
5	Karan Mehta	Support Engineer	1	null	null
6	Priya Sharma	Developer	1	null	null
7	Neha Kapoor	HR Executive	2	8	Rahul Jain
8	Rahul Jain	HR Manager	2	8	Rahul Jain
9	Simran Kaur	Recruiter	2	8	Rahul Jain
10	Aditya Roy	HR Executive	2	8	Rahul Jain
11	Megha Joshi	Recruiter	2	8	Rahul Jain
12	Vivek Malhotra	HR Manager	2	8	Rahul Jain
13	Ishita Rao	Accountant	3	18	Sahil Gupta
14	Manish Kumar	Financial Analyst	3	18	Sahil Gupta
15	Pooja Agarwal	Accountant	3	18	Sahil Gupta
16	Rohit Bansal	Financial Analyst	3	18	Sahil Gupta
17	Kavya Nair	Accountant	3	18	Sahil Gupta
18	Sahil Gupta	Finance Manager	3	18	Sahil Gupta
19	Tanya Singh	Marketing Executive	4	21	Nisha Patel
20	Yash Verma	Marketing Analyst	4	21	Nisha Patel
21	Nisha Patel	Marketing Manager	4	21	Nisha Patel
22	Aman Khan	Sales Executive	4	21	Nisha Patel
23	Shreya Das	Marketing Executive	4	21	Nisha Patel
24	Dev Sharma	Sales Executive	4	21	Nisha Patel
25	Akash Yadav	Operations Executive	5	27	Varun Singh
26	Sneha Roy	Operations Analyst	5	27	Varun Singh
27	Varun Singh	Operations Manager	5	27	Varun Singh
28	Komal Gupta	Operations Executive	5	27	Varun Singh
29	Nitin Sharma	Process Analyst	5	27	Varun Singh
30	Aditi Mehta	Operations Manager	5	27	Varun Singh

4. Test View Updatability

A view is **updatable** when modifications made through the view can be translated unambiguously to the underlying table.

Test 1 — Update Employee Hierarchy View

Try:

UPDATE Employee_Hierarchy

SET Employee_Name = 'Aarav Kumar'

WHERE Emp_ID = 1;

This view contains a **self-join** between two instances of Employee, so it is not a simple single-table view. MySQL will generally reject modifications to this view because it is not inherently updatable.

Error: The target table Employee_Hierarchy of the UPDATE is not updatable

Therefore, use:

SELECT *

FROM Employee_Hierarchy

WHERE Emp_ID = 1;

to verify the data instead.

Emp_ID	Employee_Name	Job_Title	Dept_ID	Manager_ID	Manager_Name
1	Aarav Sharma	Developer	1	null	null

5. Test Department Salary Summary View

Try:

```
UPDATE Department_Salary_Summary
```

```
SET Average_Salary = 60000
```

```
WHERE Dept_ID = 1;
```

This view is **not updatable** because it contains:

- GROUP BY
- aggregate functions such as SUM(), AVG(), COUNT(), MIN(), and MAX()
- a LEFT JOIN

The value Average_Salary is calculated from multiple employee rows, so MySQL cannot determine which individual employee record should be changed.

Error: The target table Department_Salary_Summary of the UPDATE is not updatable

6. Create a Simple Updatable View

To demonstrate an actually updatable view, create a simple view containing rows directly from Employee.

```
CREATE VIEW Employee_Salary_View AS
```

```
SELECT
```

```
Emp_ID, Emp_Name, Salary, Dept_ID
```

```
FROM Employee;
```

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

View the data:

```
SELECT *
```

```
FROM Employee_Salary_View;
```

Emp_ID	Emp_Name	Salary	Dept_ID
1	Aarav Sharma	55000.00	1
2	Riya Gupta	62000.00	1
3	Arjun Verma	48000.00	1
4	Ananya Singh	70000.00	1
5	Karan Mehta	45000.00	1
6	Priya Sharma	58000.00	1
7	Neha Kapoor	50000.00	2
8	Rahul Jain	65000.00	2
9	Simran Kaur	42000.00	2
10	Aditya Roy	46000.00	2
11	Megha Joshi	55000.00	2
12	Vivek Malhotra	72000.00	2
13	Ishita Rao	60000.00	3
14	Manish Kumar	75000.00	3
15	Pooja Agarwal	52000.00	3
16	Rohit Bansal	68000.00	3
17	Kavya Nair	58000.00	3
18	Sahil Gupta	80000.00	3
19	Tanya Singh	47000.00	4
20	Yash Verma	56000.00	4
21	Nisha Patel	62000.00	4
22	Aman Khan	44000.00	4
23	Shreya Das	51000.00	4
24	Dev Sharma	59000.00	4
25	Akash Yadav	49000.00	5
26	Sneha Roy	57000.00	5
27	Varun Singh	63000.00	5
28	Komal Gupta	46000.00	5
29	Nitin Sharma	54000.00	5
30	Aditi Mehta	69000.00	5

Now update an employee through the view:

UPDATE Employee_Salary_View

SET Salary = 65000

WHERE Emp_ID = 1;

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	1	0	Rows matched: 1 Changed: 1 Warnings: 0	2	0	1

Check the original table:

SELECT Emp_ID, Emp_Name, Salary

FROM Employee

WHERE Emp_ID = 1;

Emp_ID	Emp_Name	Salary
1	Aarav Sharma	65000.00

7. Recursive CTE

A **recursive Common Table Expression (CTE)** is useful for hierarchical data such as:

The recursive CTE has two parts:

1. **Anchor query** — finds the starting/top-level employees.
2. **Recursive query** — repeatedly finds employees reporting to the previous level.

8. Display Complete Reporting Chains

WITH RECURSIVE EmployeeChain AS

(

-- Anchor member

SELECT

Emp_ID, Emp_Name, Manager_ID, 0 AS Level,

CAST(Emp_Name AS CHAR(1000)) AS Reporting_Chain

FROM Employee

WHERE Manager_ID IS NULL

UNION ALL

-- Recursive member

SELECT

e.Emp_ID, e.Emp_Name, e.Manager_ID, ec.Level + 1,

CONCAT(ec.Reporting_Chain, ' -> ', e.Emp_Name)

FROM Employee e

INNER JOIN EmployeeChain ec

ON e.Manager_ID = ec.Emp_ID

)

SELECT

Emp_ID, Emp_Name, Manager_ID, Level,

Reporting_Chain

FROM EmployeeChain

ORDER BY Reporting_Chain;

Emp_ID	Emp_Name	Manager_ID	Level	Reporting_Chain
1	Aarav Sharma	null	0	Aarav Sharma
4	Ananya Singh	null	0	Ananya Singh
3	Arjun Verma	null	0	Arjun Verma
5	Karan Mehta	null	0	Karan Mehta
6	Priya Sharma	null	0	Priya Sharma
2	Riya Gupta	null	0	Riya Gupta

9. Display Hierarchy Level by Level

If you only want to see the employee's hierarchy level:

WITH RECURSIVE EmployeeHierarchy AS

(

SELECT

Emp_ID, Emp_Name, Manager_ID, 0 AS Hierarchy_Level

```
FROM Employee
WHERE Manager_ID IS NULL
```

```
UNION ALL
```

```
SELECT
```

```
    e.Emp_ID, e.Emp_Name, e.Manager_ID, eh.Hierarchy_Level + 1
```

```
FROM Employee e
```

```
JOIN EmployeeHierarchy eh
```

```
    ON e.Manager_ID = eh.Emp_ID
```

```
)
```

```
SELECT
```

```
    Emp_ID, Emp_Name, Manager_ID, Hierarchy_Level
```

```
FROM EmployeeHierarchy
```

```
ORDER BY Hierarchy_Level, Emp_ID;
```

Emp_ID	Emp_Name	Manager_ID	Hierarchy_Level
1	Aarav Sharma	null	0
2	Riya Gupta	null	0
3	Arjun Verma	null	0
4	Ananya Singh	null	0
5	Karan Mehta	null	0
6	Priya Sharma	null	0

10. Find All Employees Reporting to One Manager

For example, find the complete hierarchy below employee 4:

WITH RECURSIVE Subordinates AS

```
(
```

```
    SELECT
```

```
        Emp_ID, Emp_Name, Manager_ID, 0 AS Level
```

```
FROM Employee
```

```
WHERE Emp_ID = 4
```

```
UNION ALL
```

```
SELECT
```

```
    e.Emp_ID, e.Emp_Name, e.Manager_ID, s.Level + 1
```

```
FROM Employee e
```

```
JOIN Subordinates s
```

```
    ON e.Manager_ID = s.Emp_ID
```

```
)
```

```
SELECT *
```

```
FROM Subordinates;
```

Emp_ID	Emp_Name	Manager_ID	Level
4	Ananya Singh	null	0